

Ejercicio 22: Camino de Suma Mínima en un Triángulo (Programación Dinámica)

Enunciado

Dada una matriz triangular, encontrar el camino de suma mínima de arriba a abajo, desde la fila 1 y columna 1. En cada paso podemos movernos a las posiciones adyacentes de la fila siguiente: si estamos en la columna j de la fila i , podemos ir a la columna j o a la $j + 1$ de la fila $i + 1$.

1. Ecuación en Recurrencias

Para evitar condiciones de parada complejas e índices fuera de rango en los extremos laterales del triángulo, utilizaremos una formulación **Bottom-Up (de abajo hacia arriba)**.

Definición del Estado

Definimos la función óptima de estado como:

- $DP[i][j]$: El coste mínimo (suma mínima acumulada) para viajar desde la posición (i, j) del triángulo hasta la base (fila n).

Relación de Recurrencia

Si estamos en la celda (i, j) de la fila i , para continuar nuestro camino hacia la base debemos elegir movernos de manera óptima a una de las dos celdas adyacentes permitidas en la fila de abajo: la celda $(i + 1, j)$ o la celda $(i + 1, j + 1)$.

$$DP[i][j] = M[i][j] + \min(DP[i + 1][j], DP[i + 1][j + 1])$$

Donde $M[i][j]$ es el valor del elemento en la posición (i, j) del triángulo.

Casos Base (La base del triángulo)

Para la última fila n , el coste de llegar a la base es simplemente el valor de la propia celda, ya que no podemos descender más:

$$DP[n][j] = M[n][j] \quad \forall 1 \leq j \leq n$$

Al calcular de abajo hacia arriba, el resultado del coste mínimo global del triángulo se encontrará directamente en la celda inicial: $DP[1][1]$.

2. Diseño del Algoritmo (Bottom-Up)

Rellenamos la tabla de forma iterativa ascendente, desde la fila $n - 1$ hasta la fila 1.

Para poder reconstruir el camino exacto realizado de arriba a abajo, utilizaremos una matriz auxiliar `siguiente[i][j]` que registrará si en cada paso la decisión óptima fue ir recto hacia abajo (columna j , que guardaremos como 0) o en diagonal (columna $j + 1$, que guardaremos como 1).

```

Algoritmo CaminoMinimoTriangulo(M, n)
    // Entrada:
    //   - M[1..n][1..n]: Matriz triangular (con ceros en la zona no utilizada)
    //   - n: Número de filas del triángulo
    // Salida:
    //   - La suma mínima y el camino de celdas recorrido

    Crear matriz DP[1..n][1..n]
    Crear matriz siguiente[1..n-1][1..n-1]

    // 1. Inicializar Caso Base (Última fila de la matriz DP)
    Para j ← 1 hasta n hacer
        DP[n][j] ← M[n][j]
    FinPara

    // 2. Rellenar la tabla iterativamente de abajo hacia arriba
    Para i ← n - 1 decreciendo hasta 1 hacer
        Para j ← 1 hasta i hacer

            // Evaluamos las dos celdas adyacentes de la fila de abajo
            opcion_abajo ← DP[i+1][j]
            opcion_diagonal ← DP[i+1][j+1]

            Si (opcion_abajo <= opcion_diagonal) entonces
                DP[i][j] ← M[i][j] + opcion_abajo
                siguiente[i][j] ← 0 // Decisión: Ir a la columna j (abajo)
            Sino
                DP[i][j] ← M[i][j] + opcion_diagonal
                siguiente[i][j] ← 1 // Decisión: Ir a la columna j+1 (diagonal)
            FinSi

        FinPara
    FinPara

    // 3. Reconstrucción del camino óptimo
    Escribir "Suma Mínima Obtenida: ", DP[1][1]
    Escribir "Camino recorrido:"

    col_actual ← 1
    Para i ← 1 hasta n - 1 hacer
        Escribir "Fila ", i, ", Columna ", col_actual, " (Valor: ", M[i][col_actu

        // Aplicar la decisión guardada para avanzar a la siguiente fila
        decision ← siguiente[i][col_actual]
        col_actual ← col_actual + decision
    FinPara

    // Escribir el último nodo en la base
    Escribir "Fila ", n, ", Columna ", col_actual, " (Valor: ", M[n][col_actual],

    Retornar DP[1][1]
FinAlgoritmo

```

3. Análisis de Eficiencia

número total de estados (celdas de la matriz DP) que calculamos es proporcional al número de elementos del triángulo de tamaño n :

$$\text{Celdas calculadas} = 1 + 2 + \dots + n = \frac{n(n+1)}{2} \in \mathcal{O}(n^2)$$

Como cada celda se procesa en tiempo constante $\mathcal{O}(1)$ realizando una comparación de mínimo y una suma, el coste temporal global es estrictamente de $\mathcal{O}(n^2)$. Esto es asintóticamente óptimo dado que cualquier algoritmo debe al menos leer las celdas de entrada del triángulo.

- **Complejidad Espacial:** $\mathcal{O}(n^2)$. Necesitamos almacenar las matrices DP y siguiente de dimensiones $n \times n$.

4. Solución Espacialmente Optimizada ($\mathcal{O}(n)$ en Espacio)

Si en el examen **solo te piden calcular la suma mínima** y no necesitas reconstruir el camino explícitamente, puedes optimizar el uso de memoria de manera brillante.

Dado que para calcular la fila i únicamente necesitamos los datos de la fila de abajo $i + 1$, podemos realizar la actualización utilizando **un único vector unidimensional** de tamaño n que se va sobrescribiendo sobre sí mismo de abajo hacia arriba:

Pseudocódigo Optimizado

```
Algoritmo SumaMinimaOptimizada(M, n)
    // Entrada: Matriz triangular M[1..n][1..n], número de filas n
    // Salida: La suma mínima de arriba a abajo sin usar matrices adicionales

    Crear vector DP[1..n]

    // Inicializar el vector con los valores de la base del triángulo
    Para j ← 1 hasta n hacer
        DP[j] ← M[n][j]
    FinPara

    // Sobrescribir el vector fila a fila de abajo hacia arriba
    Para i ← n - 1 decreciendo hasta 1 hacer
        Para j ← 1 hasta i hacer
            DP[j] ← M[i][j] + Minimo(DP[j], DP[j+1])
        FinPara
    FinPara

    // El resultado final se concentra en la primera posición del vector
    Retornar DP[1]
FinAlgoritmo
```

Eficiencia de la Solución Optimizada:

- **Tiempo:** $\mathcal{O}(n^2)$ (mismo número de cálculos).
- **Espacio:** $\mathcal{O}(n)$ (el vector unidimensional DP consume únicamente memoria lineal, una mejora drástica frente al espacio matricial).